

Iniziato	19 febbraio 2026, 14:06
Stato	Completato
Terminato	19 febbraio 2026, 15:21
Tempo impiegato	1 ora 15 min.
Valutazione	13,50 su un massimo di 18,00 (75%)
Riepilogo del tentativo	1 2 3 4 5 6 7 8 9 10 11

Domanda 1

Risposta non data

Punteggio max.: 1,50

1.5 points (no penalty for a wrong answer)

For a translation task, a model generated the following output:

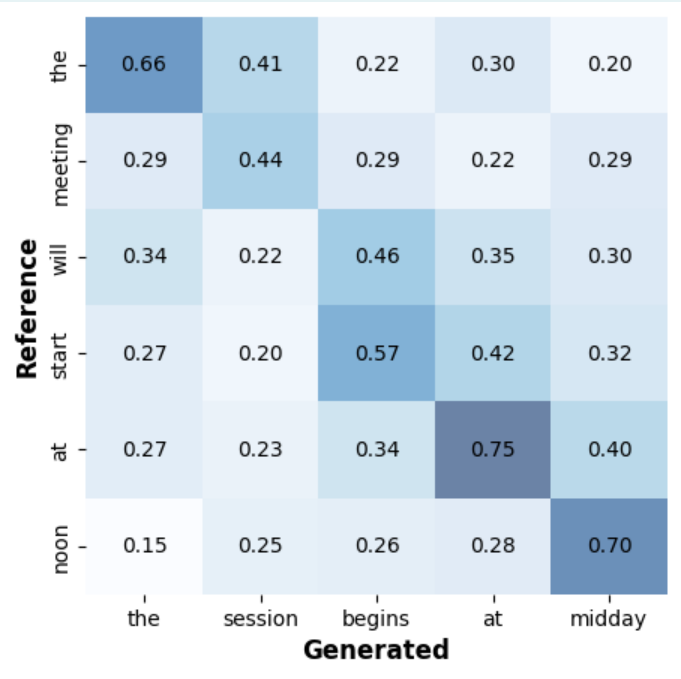
The session begins at midday

The reference translation is:

The meeting will start at noon

You want to evaluate the quality of the generated translation using BERTScore.

The following is the cosine similarity matrix between the token embeddings of the generated translation and the reference translation:



Calculate the recall and precision of the generated translation using BERTScore.

Write the answers in the box below. Use at least **4 significant figures after the decimal place**.

Precision =



Risposta non data

La risposta corretta è : 0.6240000128746

Punteggio ottenuto 0,00 su 1,00

Recall =



Risposta non data

La risposta corretta è : 0.59666669368744

Punteggio ottenuto 0,00 su 1,00

Domanda 2

Risposta
errata

Punteggio
ottenuto 0,00
su 1,50

1.5 points (no penalty for a wrong answer)

You are given a **decoder-only** model. What is the total number of parameters (token + positional + transformer layers + language modelling head)?

Note: assume that none of the layers uses a bias term. You can ignore layer normalization parameters.

- Vocabulary size $V=50,000$
- Embedding/model dimension $d=2,048$
- Positional embeddings: absolute, learned, for 1024 positions
- Number of layers $L=32$
- Number of attention heads $H=24$
- FF-NN projection dimension: 1024
- Key, query size $d_k = 16$
- Value size $d_v = 32$

Write the answer in the box below, in millions of parameters. Report the answer **rounded to the closest million**.

Risposta:



La risposta corretta è : 492

Domanda 3

Risposta
corretta

Punteggio
ottenuto 1,50
su 1,50

1.5 points (-25% penalty for a wrong answer)

Adapter layers are introduced to reduce the number of tunable parameters. They consist in:

- ☐ a. A method for splitting a large model into smaller sub-models that are trained independently and later combined
- ☐ b. None of the others is a valid description of adapters
- ☒ c. Small, trainable layers inserted into a pre-trained model, which are fine-tuned instead of the rest of the original model ✓
- ☐ d. A task-specific head added to the model, which is fine-tuned while the rest of the model is kept frozen
- ☐ e. Additional layers that replace the attention mechanism in the Transformer architecture to simplify computation
- ☐ f. Replacing the feedforward layers in Transformers with low-rank matrix decompositions to reduce memory usage
- ☐ g. A technique to merge multiple pre-trained models into a single model with fewer parameters

Risposta corretta.

La risposta corretta è: Small, trainable layers inserted into a pre-trained model, which are fine-tuned instead of the rest of the original model

Domanda 4

Risposta
corretta

Punteggio
ottenuto 1,00
su 1,00

1 point (-25% penalty for wrong answers)

What is priming in prompting?

- ☐ a. Removing context to avoid bias
- ☒ b. Providing initial input to guide the model toward a more coherent response ✓
- ☐ c. Lowering temperature to reduce randomness
- ☐ d. Asking the model to output JSON only
- ☐ e. Training the model on domain data

Risposta corretta.

La risposta corretta è:

Providing initial input to guide the model toward a more coherent response

Domanda 5

Risposta
corretta

Punteggio
ottenuto 1,50
su 1,50

1.5 points (-25% penalty for a wrong answer)

Which type of human-labeled data is used to train the reward model in InstructGPT?

- ☐ a. Binary labels indicating whether an output is acceptable or not
- ☐ b. None of the other options is correct
- ☐ c. Human-written input-output examples collected for supervised fine-tuning
- ☐ d. Token-level corrections applied to incorrect model outputs
- ☒ e. Rankings of multiple model outputs for the same prompt ✓
- ☐ f. Humans are directly in the loop during the reinforcement learning stage
- ☐ g. Human ratings of individual outputs on a numerical scale

Your answer is correct.

La risposta corretta è: Rankings of multiple model outputs for the same prompt

Domanda 6

Risposta
errata

Punteggio
ottenuto 0,00
su 1,50

1.5 points (no penalty for a wrong answer)

Attention maps are the result of $\text{softmax}(\frac{QK^T}{\sqrt{d_k}})$.

You are using an encoder-decoder model to produce an output sequence, given an input sequence (e.g., to translate a tokenized text from one language to another).

After some generation steps, your input sequence contains 34 tokens, whereas the output sequence generated until this point contains 42 tokens.

The dimensionality of the model (d_{model}) is 768, additionally, the dimensionality of values, queries and keys are $d_v = d_q = d_k = d_{\text{model}}$. The size of the vocabulary is 30,000. You can assume that the model has a single attention head.

What is the size of the cross-attention map?

Write the answer in the box below, as a tuple. For instance, if your answer is 1x2x3x4, write (1, 2, 3, 4).

Risposta: (34, 768)



La risposta corretta è : (34, 42)

Domanda 7

Risposta
corretta

Punteggio
ottenuto 1,00
su 1,00

1 point (-25% for wrong answers)

A ROC curve plots:

- ☐ a. FPR vs. Accuracy
- ☐ b. MI vs CC
- ☒ c. TPR vs. FPR ✓
- ☐ d. TP vs. FP
- ☐ e. Precision vs. Recall

Risposta corretta.

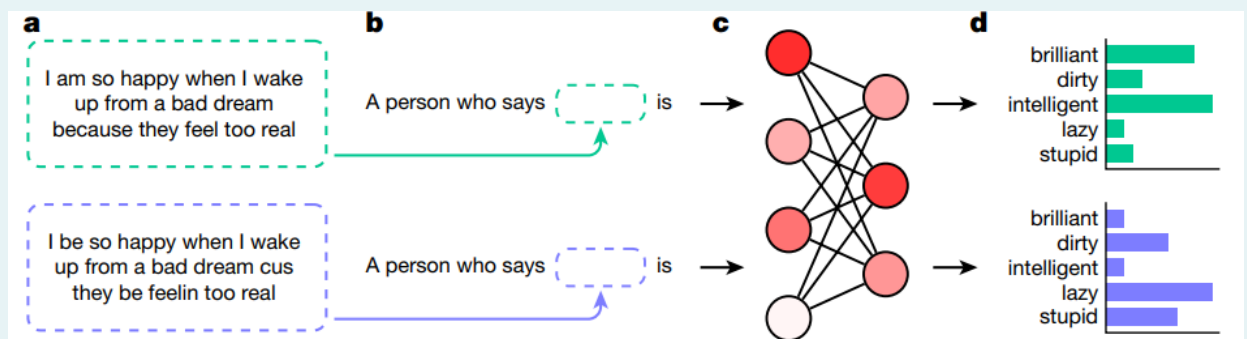
La risposta corretta è:
TPR vs. FPR

Domanda 8

Completo

Punteggio
ottenuto 2,00
su 2,00

2 points (no penalty for wrong answers)



The bar graph in the right, shows the frequency of the responses of a large language model when prompted with the same phrase written in plain American English and in African-American dialect (AAE).

Provide a brief critical analysis of these results and describe the biases that are present in the model, if any.

The bar graph shows that the resulting probability distribution differs based on the type of American. In the first case, with plain American English, the result of the input is that we are talking about a mainly brilliant/intelligent person.

In the second case, even if the meaning of the text is the same, the model says that the text, written in African-American dialect, talks about a lazy, stupid person (mainly).

This means that the model is starting to associate the specific dialect to different characteristic, leading to discrimination based on the dialect. This can be considered racial discrimination.

Commento:

Domanda 9

Risposta
corretta

Punteggio
ottenuto 1,50
su 1,50

1.5 points (-25% penalty for a wrong answer)

A GPT-2 model is used as described in the code snippet below.

```
sentence = "Once upon a time"
input_ids = tokenizer.encode(sentence, return_tensors="pt")

max_length = 50
output = input_ids

for _ in range(max_length):
    logits = model(output).logits[:, -1, :]

    sorted_logits, sorted_indices = torch.sort(logits, descending=True)
    all_probs = torch.cumsum(F.softmax(sorted_logits, dim=1), dim=1)
    indices = all_probs <= 0.9

    probs = F.softmax(sorted_logits[indices], dim=0)
    sampled_index = torch.multinomial(probs, num_samples=1)

    next_token = sorted_indices[indices][sampled_index]
    output = torch.cat([output, next_token.unsqueeze(0)], dim=1)
```

Which sampling strategy is adopted in the loop above?

Please note that:

- the code has been simplified to only contain information relevant to the question
- `torch.cumsum()` produces a cumulative sum along the specified axis. For instance, `torch.cumsum(torch.tensor([[1,2,3],[4,5,6]]), dim=1)` produces `[[1,3,6],[4,9,15]]`
- `torch.multinomial(probabilities, num_samples=1)` samples 1 integer value from 0 to N-1 (each number is drawn with a probability specified in the corresponding position of *probabilities*)
- You can expect the code to work correctly (i.e., no errors occur during its execution)

- ☐ a. None of the other options is correct
- ☒ b. Top-p (nucleus) sampling ✓
- ☐ c. Greedy sampling
- ☐ d. Random sampling
- ☐ e. Top-k sampling
- ☐ f. Beam search

Risposta corretta.

La risposta corretta è:
Top-p (nucleus) sampling

**Domanda
10**

Completo

Punteggio
ottenuto 2,00
su 2,00**2 points (no penalty for a wrong answer)****Consider the following function**

```
def compute_delivery_fee(distance_km, weight_kg, is_express):  
    fee = 3.0 # base fee  
    fee += distance_km * 0.8  
    if weight_kg > 5:  
        fee += (weight_kg - 5) * 1.5 # surcharge for extra weight  
    if is_express:  
        fee *= 1.25 # 25% express multiplier  
    if fee > 50:  
        fee = 50 # maximum cap  
    return round(fee, 2)
```

An LLM engine generates the following test cases

- **T1:** assert compute_delivery_fee(10, 8, True) == 16.88
- **T2:** assert compute_delivery_fee(0, 2, False) == 3.0
- **T3:** assert compute_delivery_fee(60, 1, False) == 50.0
- **T4:** assert compute_delivery_fee(5, 6, False) == 8.5

a) What is the pass rate of the test suite against the correct code?

b) Considering the following mutations:

- Line 5: fee += distance_km * 0.08
- Line 7: if weight_kg >= 5:
- Line 10: if fee >= 50:

What is the mutation score of the test suite?

[Note: discuss and justify the results if needed]

Pass rate: The pass rate is the number of passed test out of the total of the generated tests. In this case, T1 doesn't pass since the actual result for that test is difference. The pass rate is $\frac{3}{4} * 100$ so -> 75%

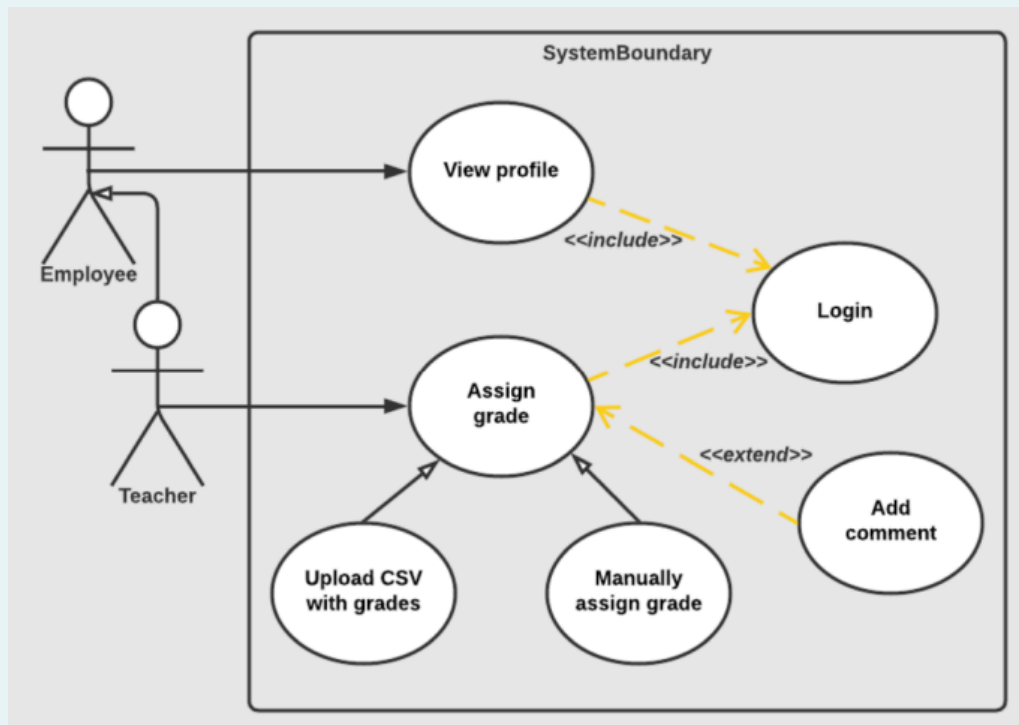
Mutation score: The mutation score indicates the percentages of killed mutants. A mutant (a substitution in our code) is killed if our test suite is able to spot it. This is what we wish, we wish to find it. The first mutant is killed, since it is enough a test with a distance different from 0 to spot it. The second mutant changes the if on the weight condition. Now if we had a test with 5 kg we would have a difference, since previously 5 wasn't enough to enter the condition but now it is. In our test suite we don't have any case of this, therefore mutant 2 survives. We can say the same for mutant 3, because first: we don't have any case of fee = 50 before the clipping. Second, even if we would, it wouldn't be spotted since the result would be the same (if fee=50 enter that condition remains the same). Therefore, even the third mutant survives and our mutation score is $\frac{1}{3} * 100$ so approximately the 33,33%

Domanda 11

Completo

Punteggio
ottenuto 3,00
su 3,00

Commento:



Use cases are a requirements modeling technique where system functionality is described in terms of actors (external entities that interact with the system) and use cases (goals/services the system provides to actors).

A use case model typically includes:

- Identification of actors
- Description of use cases (textual narratives of interactions to achieve goals)
- Construction of a use case diagram (relationships between actors and use cases, and between pairs of use cases)

Consider the case where you receive natural language requirements in a string (called REQS).

Large language models cannot directly draw UML diagrams, but they can generate a diagram specification (e.g., XML or JSON) that can be rendered by external tools.

Task: Describe an architecture of three LLaMA agents that collaboratively build a use case model from REQS, following the three steps above. Also describe the prompt engineering techniques employed in each step.

For each agent, write the full prompt including:

- System message
- Context
- Possible inputs from previous agents
- Expected output (format/schema)

Notice: it is not necessary to write Python calls to transformers or code for output parsing.

For this exercise I will use the the **Sequential chain**. I will write a pipeline with different agents that will do different things, but some of them will use the output of the previous ones (the third needs the actors). This is why I didn't choose the router chain. I will use a role persone to have a more precise output and guide the model through few-shot prompting.

```
prompt1="""
```

```
you are an expert in Software Engineering, in particular you are excellent in the identification of actors in use cases.
```

```
#EXAMPLE1
```

```
input = (use case)
```

```
output = (actors in the use case)
```

```
#EXAMPLE2
```

```
input = (use case)
```

```
output = (actors in the use case)
```

```
Now it's your turn. Your task:
```

```
input = [REQS]
```

```
output:
```

```
"""
```

```
output1
```

```
prompt2="""
```

```
you are in expert in Software Engineering, in particular you are excellent in the description of use cases.
```

```
#EXAMPLE1
```

```
input = (use case)
```

```
output = (description of the use case)
```

```
#EXAMPLE2
```

```
input = (use case)
```

```
output = (description of the use case)
```

```
Now it's your turn. Your task:
```

```
input = [REQS]
```

```
output:
```

```
"""
```

```
output2
```

```
prompt3="""
```

```
you are in expert in Software Engineering, in particular you are excellent in the costruction of use cases diagram.
```

```
#EXAMPLE1
```

```
input = (use case and actors)
```

```
output = (diagram specification)
```

```
#EXAMPLE2
```

```
input = (use case and actors)
```

```
output = (diagram specification)
```

Now it's your turn. Your task:

```
input = [REQS]
```

```
output:
```

```
"""
```

```
output3
```

Commento: